Deep Convolutional Neural Networks for Semantic Segmentation

Merrick Qiu
UC San Diego
La Jolla, CA 92093
myqiu@ucsd.edu

Michael Ye
UC San Diego
La Jolla, CA 92093
mhye@ucsd.edu

Collin Cranston
UC San Diego
La Jolla, CA 92093
ccransto@ucsd.edu

Abstract

We used a deep convolutional neural network in order to perform semantic segmentation on the PASCAL VOC 2012 dataset. We built a PyTorch CNN with ReLU activation function, batch normalization, and an encoder and decoder architecture. With the baseline model we were able to get a mean pixel accuracy of 83.536% and mean IoU score of 0.24001 but after weighting classes and applying learning rate scheduling we were able to get a model with a mean pixel accuracy of 84.398% and a mean IoU score of 0.25238. While we experimented with a variety of techniques to improve the baseline, only the learning rate scheduler and weighted loss ultimately improved our model performance. Data Augmentation failed to improve our model, which we attributed to an insuffient number of epochs during training.

1 Introduction

1.1 Overview

Semantic segmentation is the process by which the pixels in an image are classified into a number of categories. In the PASCAL VOC 2012 dataset, each pixel is classified into one of 21 classes using a softmax. This is useful in a number of fields including self-driving cars, medical imaging, and satellite imaging. However, this is a challenging task because it requires the model to understand the context of the image and to be able to distinguish between different objects.

To perform semantic segmentation, we used a deep convolutional neural network with an encoder and decoder architecture. Convolutions can calculate the features of an image by repeatedly applying a filter to an image. We use ReLU as our acitvation function to add nonlinearity to the network. Batch normalization normalizes the input to each layer so that it has zero mean and scaling it to the standard deviation. This speeds up training and makes the model more robust. To perform a feedforward pass, the encoder is used to extract features from the image and the decoder is used to upsample the features to the original image size.

1.2 Xavier Weight Initialization

The output from a matrix multiplication has a variance equal to the the size of the input vector if the weights and inputs are all initated with standard normal distributions. In addition, the activation function can change the variance of the output. In order to prevent activations from exploding or

Preprint. Under review.

vanishing during the forward pass of the initial model, the weights are set using Xavier weight initialization.

We want the variance of the layer outputs to be 1, which approximately happens when the weight values are chosen from the uniform distribution $\mathcal{U}(-\sqrt{6/(n_i + n_{i+1})}, \sqrt{6/(n_i + n_{i+1})})$, where n_i is the fan-in and n_{i+1} is the fan-out. This distribution comes from research done by Glorot and Bengio (2010) [1].

2 Related Work

The steps used in the model are inspired by techniques found by previous researchers. Convolutional neural networks were popularized by the paper by Yann LeCun et al. (1989) [4] on handwritten zip code recognition. As already stated, Xavier weight initialization is used as defined in the paper by Glorot and Bengio (2010) [1]. Batch normalization was first proposed by Sergey Ioffe and Christian Szegedy (2015)[3]. Finally, the use of a convolutional neural network for image segmentation has also been done by Olaf Ronneberger et al. (2015) with their U-Net architecture [5] and by Kaiming He et al. (2018) [2] with their Mask R-CNN architecture.

3 Methods

3.1 Basline Model architecture

The architecture of the basline model is shown in table 1.

Table 1: Baseline model architecture

Type	Output Channels	Kernel Size / Stride / Padding	Activation Function
Conv2d	32	3x3 / 2 / 1	ReLU
BatchNorm2d	32	-	-
Conv2d	64	3x3 / 2 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d	128	3x3 / 2 / 1	ReLU
BatchNorm2d	128	-	-
Conv2d	256	3x3 / 2 / 1	ReLU
BatchNorm2d	256	-	-
Conv2d	512	3x3 / 2 / 1	ReLU
BatchNorm2d	512	-	-
ConvTranspose2d	512	3x3 / 2 / 1 / 1	ReLU
BatchNorm2d	512	-	-
ConvTranspose2d	256	3x3 / 2 / 1 / 1	ReLU
BatchNorm2d	256	-	-
ConvTranspose2d	128	3x3 / 2 / 1 / 1	ReLU
BatchNorm2d	128	-	-
ConvTranspose2d	64	3x3 / 2 / 1 / 1	ReLU
BatchNorm2d	64	-	-
ConvTranspose2d	32	3x3 / 2 / 1 / 1	ReLU
BatchNorm2d	32	-	-
Conv2d(Classifier)	n_class	1x1 / - / -	-

3.2 Experimental Architecture

3.2.1 Experimental Architecture I

The following experimental architecture tries to integrate fully connected layers into the model. However, the model did not perform as well as the baseline model and took much longer to train due to the extra weights. The model early stopped at around 7 epochs and is only able to obtain an accuracy of 0.75 and IoU of 0.056.

Table 2: Experimental architecture 1

Type	Output Channels	Kernel Size / Stride / Padding	Activation Function
Conv2d	64	3x3 / 1 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d(Fully Connected)	64	1x1 / 1 / 1	-
Conv2d	64	3x3 / 1 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d(Fully Connected)	64	1x1 / 1 / 1	-
Conv2d	64	3x3 / 1 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d(Fully Connected)	64	1x1 / 1 / 1	-
Conv2d	64	3x3 / 1 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d(Fully Connected)	64	1x1 / 1 / 1	-
Conv2d	64	3x3 / 1 / 1	ReLU
BatchNorm2d	64	-	-
Conv2d(Classifier)	n_class	1x1 / - / -	-

3.2.2 Experimental Architecture II

The following experimental architecture is the ResNet-50 model in the PyTorch Library. This model introduces a new term - Cfg Blocks, which are comprised of 1 Conv layer with multiple Identity layers. Cfg0 contains 1 Conv Layer with 2 Identity layers and has a $(56, 56, 64)$ input shape and $(56, 56, 256)$ output shape. Cfg1 contains 1 Conv layer with 2 Identity layers again, this time with a $(56, 56, 256)$ input shape and $(28, 28, 512)$ output shape. Cfg2 Block has 1 conv layer and 5 Identity layers with a $(28, 28, 512)$ input shape and a $(14, 14, 1024)$ output shape. Lastly, Cfg3 block contains 1 Conv Layer and 2 Identity layers with an input shape of $(14, 14, 1024)$ and an output shape of $(7, 7, 2048)$. The entire architecture is displayed below.

Table 3: Experimental architecture II

Type	Output Shape	Parameters
input	[(None, 224, 224, 3)]	0
sequential	(None, 224, 224, 3)	0
conv2d_28	64	9472
max_pooling2d	64	0
cfg0_block	64	148480
cfg1_block	64	665600
cfg2_block	64	2641920
cfg3_block	64	10526720
classifier	64	3932280

3.2.3 Experimental Architecture III

We implemented the U-Net convolutional neural network [**?**], and trained it on the VOC semantic segmentation task. See Table 4 for the architecture of the model.
Without class weights, the U-Net model quickly achieved 75% pixel accuracy, which means the model was simply predicting the background class. To counter this, we implemented class weights, to account for class imbalance and penalize the background labels with lower weights. Despite the validation error steadily decreasing across epochs, the model still experienced spikes in training error. Therefore, we also implemented the cosine annealing learning rate scheduler, and the following data augmentations: random crop, flip, and rotation.

Table 4: UNet Architecture

Type	Input/Output Channels	Kernel Size / Stride / Padding	Activation Function
ConvBlock	in/out	-	-
Conv2D	in/out	3/1/1	ReLU
BatchNorm2d	-	-	-
Conv2D	out/out	3/1/1	ReLU
BatchNorm2d	-	-	-
DownBlock	in/out	-	-
ConvBlock	in/out	-	-
MaxPool2d	out/out	-	-
UpBlock	in/out	-	-
ConvTranspose2d	in/in/2	-	-
torch.cat	-	in/2 / in	-
ConvBlock	in / out	-	-
UNet	3/ n_classes	-	-
ConvBlock	3 /64	-	-
DownBlock	64 / 128	-	-
DownBlock	128 /256	-	-
DownBlock	256 /512	-	-
DownBlock	512/ 1024	-	-
ConvBlock	1024 /1024	-	-
UpBlock	1024/ 512	-	-
UpBlock	512 /256	-	-
UpBlock	256 /128	-	-
UpBlock	128/ 64	-	-
Conv2d	64 /n_classes	-	-

4 Results

4.1 Model Metrics

Table 5: IOU and Pixel Accuracy Metrics

Model	IOU	Pixel Accuracy
Baseline	0.24001	83.536%
Learning Rate Scheduling	0.25927	83.954%
Data Augmentation	0.10226	77.803%
Weighted Classes	0.25238	84.398%

4.2 Loss Graphs

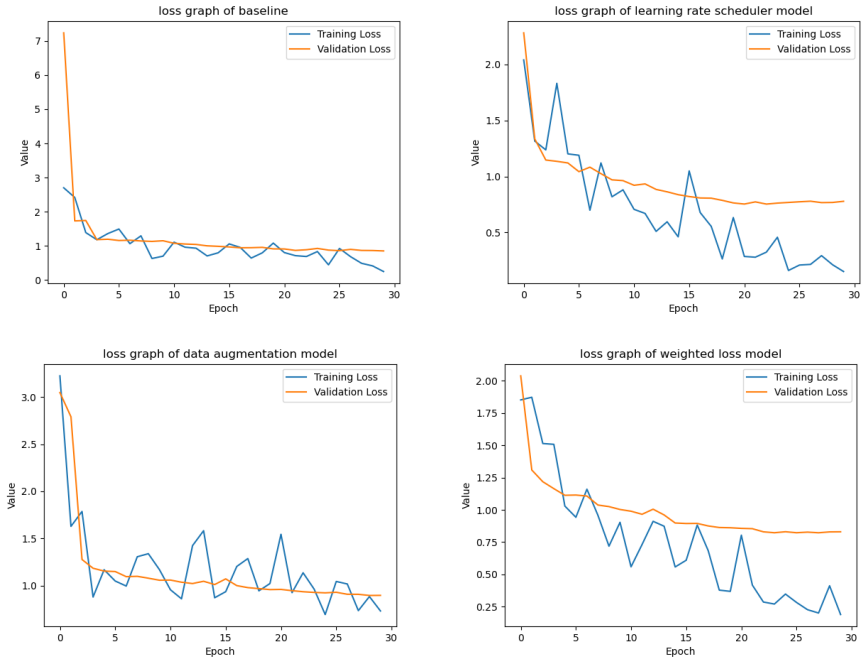

Figure 1: Training and Validation Loss of Basline, Scheduling, Data Augmentation, and Weighted Class Models

4.3 Output Visualizations

To visualize our output, we show the output masks of each model for a random image from the test set.

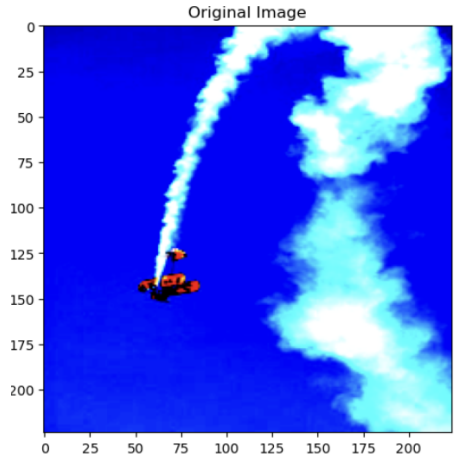

Figure 2: Original Random Test Set Image

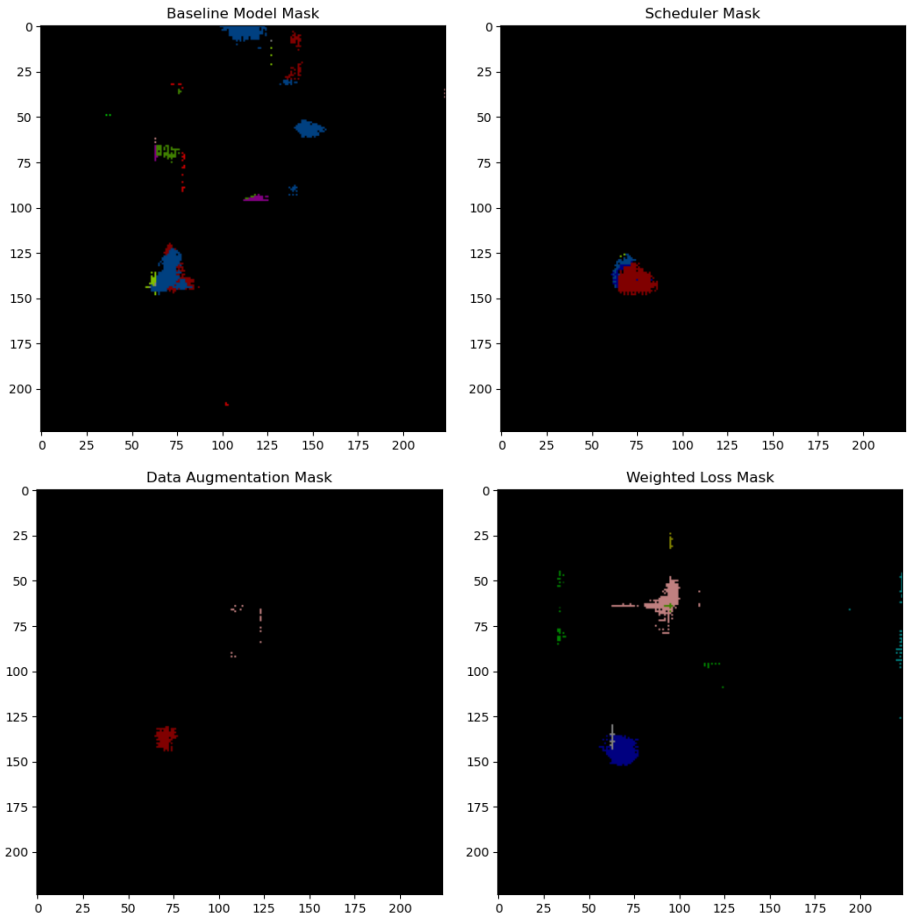

Figure 3: Output Mask of Basline, Scheduling, Data Augmentation, and Weighted Class Models

5 Discussion

5.1 Baseline Model

5.2 Improving Baseline Model

5.2.1 Cosine Annealing

5.2.2 Data Augmentation

5.2.3 Class Weights

Due to heavy class imbalances, the baseline model achieved relatively high pixel accuracy ($\sim 83.5\%$) while having a low IOU score. The model would predict the most common label (background pixel). In order to counter the class label imbalance, we implemented a power class weight function, which penalizes higher frequency labels with lower weights.

In particular, we computed class frequencies (class_freq) by counting the number of class instances in each mask, and dividing by the total number of pixels in the mask. To penalize the most frequent labels, we then inverted class_freq

$$\text{class_weight} = 1.0/\text{class_freq} \tag{1}$$

and normalized to sum to the number of classes, 21,

$$\text{class_weight} = \text{class_weight}/\text{sum}(\text{class_weight}) * \text{num_classes} \tag{2}$$

After implementing class weights, the model experienced a slight improvement over the baseline model, both in pixel accuracy and IOU. Weighted classes is a useful tool to combat class imbalance, particularly in semantic segmentation where some pixels (e.g background) can heavily dominate in training labels.

5.3 Experimentation

See Table 5 for metrics for all experimental models:

Table 6: IOU and Pixel Accuracy Metrics

Model	IOU	Pixel Accuracy
Baseline	0.24000997841358185	0.835359513759613
Alternate	0.056	0.75
Pre-trained ResNet	0.40788033604621887	0.9090900421142578
U-Net	0.10018574446439743	0.6077122688293457

5.3.1 Alternate Model

5.3.2 Transfer Learning

5.3.3 U-Net Model

Authors' Contributions

Merrick: I implemented the evaluation metrics, baseline model (including the train(), val(), modelTest() functions for the baseline model), early stopping, and helped debug with data augmentation. I also created the experimental architecture for part (a) of Experimentation and your solution. I helped write the report, especially the abstract, introduction, related work, and references sections.

Michael: I implemented the learning rate scheduler, data augmentation, train and validation loss plotting functions, transfer learning models, and data visualization. Additionally I made improvements to the model's efficiency by tuning the $num_workers$ variable to increase training speed. I wrote the results section and part of the methodology section of the report.

Collin: I implemented the U-Net model in the experimental section (5c). I also implemented the data loading for test and validation sets (voc.py), weighted class function, and helped create plots and debug. I also helped write the report, especially the methods and discussion sections.

References

[1] Xavier Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *Journal of Machine Learning Research - Proceedings Track*, 9:249–256, 01 2010.

[2] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn, 2018.

[3] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.

[4] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.